

Road Accident Analysis and Casualty Prediction Using Python

Mitali

12418121

27

Department of Computer Science & Engineering
Lovely Professional University Phagwara, Punjab
Madaanmitali720@gmail.com

Abstract—Road accidents are among the leading causes of injuries and fatalities throughout the world. Proper analysis of accident data can help identify important risk factors and support effective decision-making for traffic safety. This paper presents a road accident data analysis project developed using Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, SciPy, and Scikit-learn. A dataset containing 3000 records and 20 attributes was used for preprocessing, visualization, statistical testing, and predictive modeling. The analysis explored relationships among speed limits, alcohol involvement, weather conditions, road type, and casualties. Statistical techniques such as Shapiro-Wilk Test, Chi-Square Test, and T-Test were applied. A Linear Regression model was developed to predict the number of casualties. The study demonstrates how Python can be effectively used for real-world accident analysis and forecasting.

Keywords—Road Accidents, Python, Data Analysis, Machine Learning, Linear Regression, Statistical Testing, Visualization

I. INTRODUCTION

Road accidents create serious social, economic, and public health challenges every year. Increasing traffic volume, careless driving, overspeeding, poor road infrastructure, weather conditions, and alcohol consumption are some of the common causes of accidents. These accidents result in injuries, fatalities, property damage, and emotional loss. Therefore, studying accident data is essential to understand hidden patterns and to develop preventive strategies.

Python has become one of the most powerful programming languages for data analysis because of its simple syntax and strong libraries. In this project, Python tools were used to clean, analyze, visualize, and model road accident data. The project aims to identify major factors affecting accidents and predict casualties using machine learning techniques.

II. DATASET AND METHODOLOGY

A. Dataset Description

The dataset used in this study contains 3000 rows and 20 columns representing road accident records. It

includes variables such as State Name, Number of Casualties, Number of Fatalities, Driver Age, Speed Limit, Accident Severity, Weather Conditions, Road Type, Road Condition, Lighting Conditions, Alcohol Involvement, Driver License Status, Traffic Control Presence, and Accident Location Details. These variables provide sufficient information for descriptive, statistical, and predictive analysis.

B. Data Preprocessing

Data preprocessing was an essential step in this project. Missing values in the Traffic Control Presence column were replaced with “No Control,” while missing values in Driver License Status were replaced with “No License.” This ensured data consistency and completeness.

Code:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score

from scipy import stats
```

Data Preprocessing:

```
data = pd.read_csv(r"D:\B.tech\Sem 4\Python
Library\CA2-Project\Final Sheet.csv")

print(data.info())

data["Traffic Control Presence"] = data["Traffic
Control Presence"].fillna('No Control')

data["Driver License Status"] = data["Driver License
Status"].fillna('No License')

print(data.info())
```

C. Analytical Framework

The project used a combination of descriptive statistics, graphical visualization, hypothesis testing, and machine learning. Bar charts, pie charts, boxplots, histograms, heatmaps, and grouped charts were used for exploratory data analysis. Statistical methods such as Shapiro-Wilk Test, Chi-Square Test, and T-Test were used to validate relationships. Finally, Linear Regression was implemented to predict casualties.

III. EXPLORATORY DATA ANALYSIS

A. Objective 1: Foundational Insights – Visual Data Exploration & Relationships:

Goal

The purpose of this objective was to understand the basic structure of the dataset through summary statistics, correlation analysis, alcohol involvement distribution, and state-wise accident frequency.

Code:-

```
# (1.) Foundational Insights: Visual Data Exploration & Relationships
```

```
# Focus - Summary Statistics, Correlation, and Matplotlib.
```

```
print(data[['Number of Casualties', 'Number of Fatalities']].describe())
```

```
# Correlation Heatmap
```

```
cm = data[['Number of Casualties', 'Number of Fatalities', 'Speed Limit (km/h)', 'Driver Age']].corr()
```

```
plt.figure(figsize=(10,6))
```

```
sns.heatmap(cm, annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title("Correlation Heatmap: Road safety Variables")
```

```
plt.show()
```

```
# Alcohol Pie Chart
```

```
alcohol_counts = data['Alcohol Involvement'].value_counts()
```

```
plt.figure(figsize=(8, 8))
```

```
plt.pie(alcohol_counts, labels=alcohol_counts.index, autopct='%1.1f%%', startangle=140, colors=['skyblue', 'lightpink'], explode=(0.05, 0))
```

```
plt.title("Percentage of Total Accidents Involving Alcohol")
```

```
plt.show()
```

```
# Top 10 States
```

```
plt.figure(figsize=(12,6))
```

```
data['State
```

```
Name'].value_counts().head(10).plot(kind='bar', color='coral')
```

```
plt.title("Top 10 states by Number of Road Accidents")
```

```
plt.xlabel('State Name')
```

```
plt.ylabel('Accident Count')
```

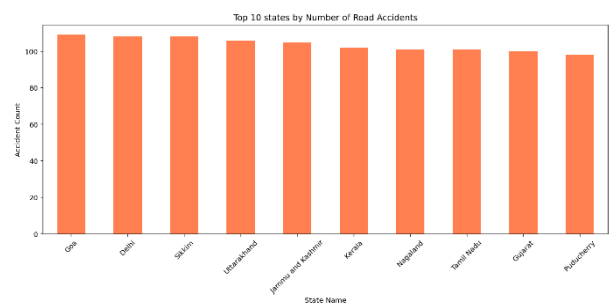
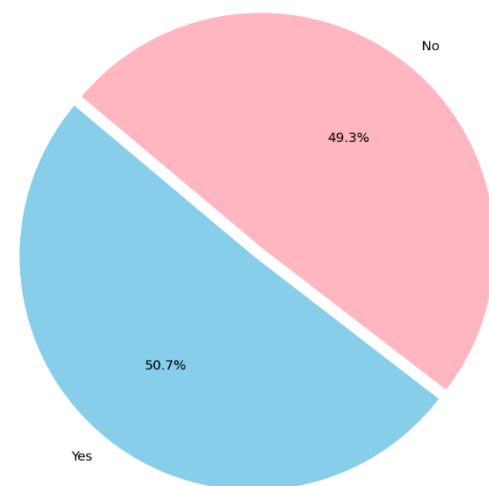
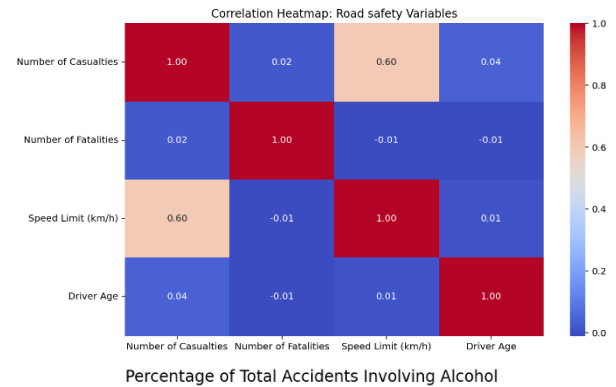
```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

OutPut:

	Number of Casualties	Number of Fatalities
count	3000.000000	3000.000000
mean	9.710667	2.455333
std	3.507286	1.717650
min	2.000000	0.000000
25%	7.750000	1.000000
50%	10.000000	2.000000
75%	12.000000	4.000000
max	59.000000	5.000000



B. Objective 2: Data Profiling – Probability Distribution & Outlier Analysis

Goal

This objective focused on examining data normality, identifying outliers, and understanding the distribution of driver age and casualties.

Code:

```
# (2.)Data Profiling: Probability Distributions &
Outlier Analysis
# Focus: Shapiro-Wilk Test, Normal Distribution, and
Anomaly Detection.
```

```
#Shapiro-Wilk Test
stat, p_value = stats.shapiro(data['Driver
Age'].head(500))
print(f'Shapiro-Wilk Test (Driver Age):
Statistics={stat:.3f},p-value={p_value:.3f}')

if p_value > 0.05:
    print("Data looks Normally Distributed (Fail to
reject Null Hypothesis)")
else:
    print("Data does not look normally distributed
(Reject Null Hypothesis)")
```

```
# Outlier Detection:
Q1 = data['Number of Casualties'].quantile(0.25)
Q3 = data['Number of Casualties'].quantile(0.75)
IQR = Q3-Q1
outliers = data[(data['Number of Casualties'] < (Q1-
1.5*IQR)) | (data['Number of
Casualties'] > (Q3+1.5*IQR))]
print(f'Number of Outliers detected in Casualties:
{len(outliers)}')
```

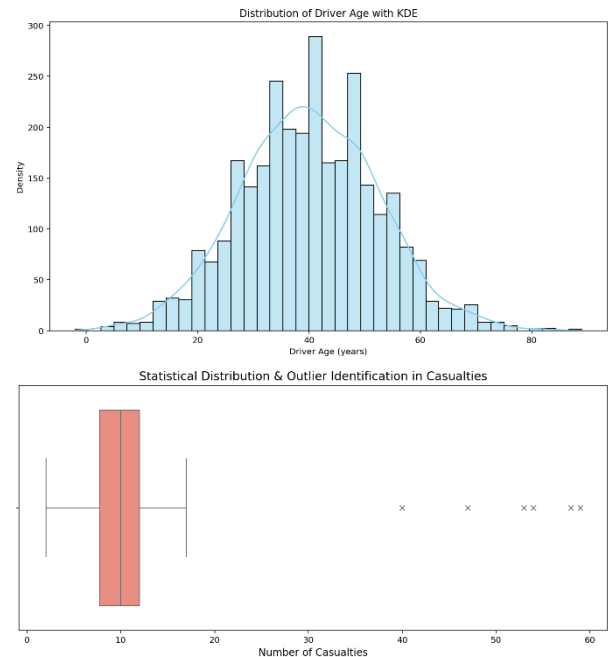
```
# Histogram
plt.figure(figsize=(10,6))
sns.histplot(data['Driver
Age'],kde=True,color='skyblue')
plt.title('Distribution of Driver Age with KDE')
plt.xlabel('Driver Age (years)')
plt.ylabel('Density')
plt.tight_layout()
plt.show()
```

```
# Casualties Boxplot
plt.figure(figsize=(10, 5))
sns.boxplot(x=data['Number of Casualties'],
color='salmon', flierprops={"marker": "x",
"markerfacecolor": "red"})
plt.title('Statistical Distribution & Outlier Identification
in Casualties', fontsize=14)
plt.xlabel('Number of Casualties', fontsize=12)
#plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
```

```
plt.show()
```

Output:

```
Shapiro-Wilk Test (Driver Age): Statistics=0.997,p-
value=0.567
Data looks Normally Distributed (Fail to reject Null
Hypothesis)
Number of Outliers detected in Casualties: 6
```

**C. Objective 3: Advanced Visuals – Statistical Storytelling with Seaborn****Goal**

The aim of this objective was to understand how environmental and infrastructure conditions influence accident casualties.

Code:

```
# (3.)Advanced Visuals: Statistical Storytelling with
Seaborn
# Focus: Advanced Seaborn plots and Customization.
```

```
# Weather vs Severity Countplot
#sns.set_theme(style="ticks")
plt.figure(figsize=(12,6))
sns.countplot(data=data,x='Weather
Conditions',hue='Accident Severity', palette="viridis")
plt.title('Accident count by weather condition and
Severity')
```

```

plt.xlabel('Weather Condition')
plt.ylabel('Total number of Accidents')
plt.legend(title='Severity', loc='upper right')
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(10, 6))
avg_cas = data.groupby('Road Type')['Number of
Casualties'].mean().sort_values()
ax = avg_cas.plot(kind='barh', color='teal')
plt.title('Infrastructure Risk: Avg Casualties by Road
Type')
plt.xlabel('Average Number of Casualties')
for i, v in enumerate(avg_cas):
    ax.text(v+0.1,i,f'{v:.2f}',va='center',fontsize=10)
plt.tight_layout()
plt.show()

```

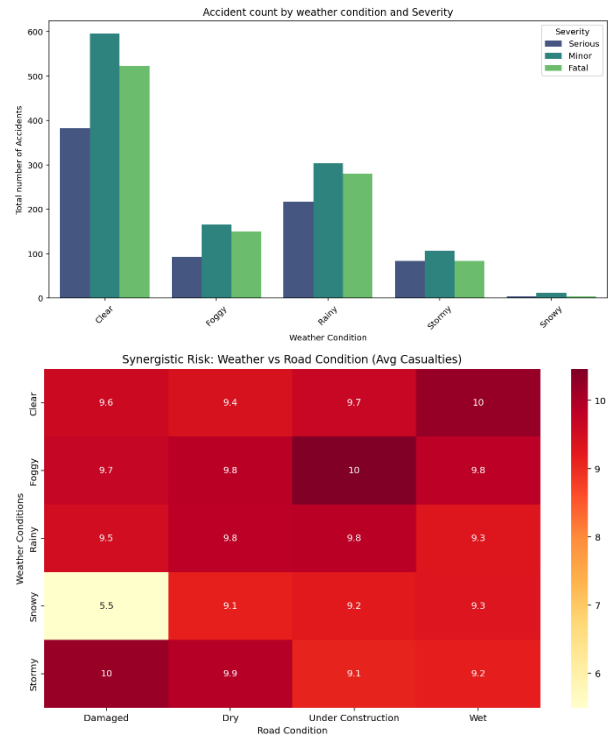
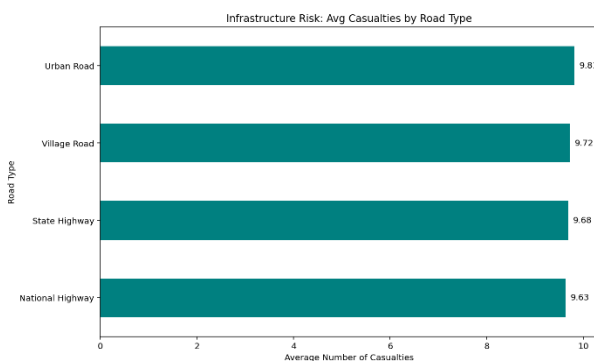
Weather vs Road Condition Heatmap

```

pivot_risk = data.pivot_table(index='Weather
Conditions', columns='Road Condition',
values='Number of Casualties', aggfunc='mean')
plt.figure(figsize=(10, 6))
sns.heatmap(pivot_risk, annot=True, cmap='YlOrRd')
plt.title('Synergistic Risk: Weather vs Road Condition
(Avg Casualties)')
plt.tight_layout()
plt.show()

```

OutPut:



D. Objective 4: Proving the Facts – Significance Testing & Safety Hypotheses

Goal

This objective was designed to statistically verify whether alcohol and speed significantly influence accident severity.

Code:

(4.) Proving the Facts: Significance Testing & Safety Hypotheses

Focus: T-Tests, Z-Tests, and Chi-Square Tests.

Chi-Square Test

```

contingency_table = pd.crosstab(data['Alcohol
Involvement'], data['Accident Severity'])
chi2, p_chi, dof, expected =
stats.chi2_contingency(contingency_table)
print(f'Chi-Square Test (Alcohol vs Severity): p-value
= {p_chi:.4f}')
if p_chi < 0.05:
    print("Alcohol has a statistically significant impact
on severity.")
else:
    print("Alcohol does not has a significant impact on
severity.")

```

T-Test

```

fatal_speed = data[data['Accident Severity'] ==
'Fatal']['Speed Limit (km/h)']

```

```

minor_speed = data[data['Accident Severity'] ==
'Minor']['Speed Limit (km/h)']
t_stat, p_test = stats.ttest_ind(fatal_speed.dropna(),
minor_speed.dropna())
print(f'T-Test (Speed of Fatal vs Minor): p-value =
{p_test:.4f}')
if p_test < 0.05:
    print("Speed significantly differs between Fatal and
Minor accidents.")
else:
    print("Speed does not differs between Fatal and
Minor accidents.")

plt.figure(figsize=(10, 6))
sns.boxplot(data=data, x='Alcohol Involvement',
y='Number of Casualties', hue='Alcohol Involvement',
palette='Set2', legend=False)
plt.title('Statistical Variance: Casualties by Alcohol
Involvement')
plt.xlabel('Alcohol Involvement')
plt.ylabel('Number of Casualties')
plt.tight_layout()
plt.show()

```

Output:

```

Chi-Square Test (Alcohol vs Severity): p-value = 0.0000
Alcohol has a statistically significant impact on severity.
T-Test (Speed of Fatal vs Minor): p-value = 0.3837
Speed does not differs between Fatal and Minor accidents.

```

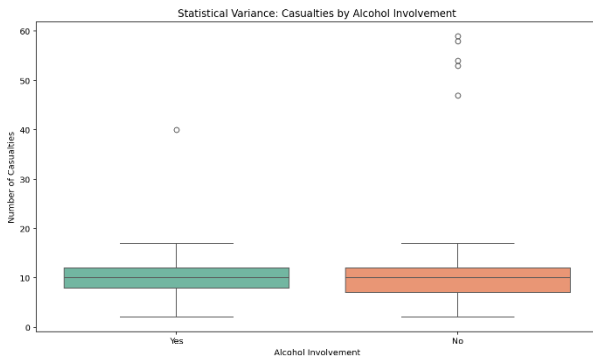


Fig. 4. Pollution vs Latitude Analysis (Objective 4).

E. Objective 5: Smart Diagnostics – Feature Quality & Experimental Design

Goal

This objective compared average casualties on National Highways and Village Roads under Daylight and Dark conditions.

Code:

```

# (5.)Smart Diagnostics: Feature Quality &
Experimental Design

# Focus: Variance Inflation Factor (VIF) and A/B
Testing.

```

```

nh_day = data[(data['Road Type'] == 'National
Highway') &
              (data['Lighting Conditions'] ==
'Daylight')]['Number of Casualties'].mean()

```

```

nh_dark = data[(data['Road Type'] == 'National
Highway') &
               (data['Lighting Conditions'] ==
'Dark')]['Number of Casualties'].mean()

```

```

vr_day = data[(data['Road Type'] == 'Village Road') &
              (data['Lighting Conditions'] ==
'Daylight')]['Number of Casualties'].mean()

```

```

vr_dark = data[(data['Road Type'] == 'Village Road') &
               (data['Lighting Conditions'] ==
'Dark')]['Number of Casualties'].mean()

```

```

print(f'National Highway in Daylight : {nh_day:.2f}')
print(f'National Highway in Dark : {nh_dark:.2f}')
print(f'Village Road in Daylight : {vr_day:.2f}')
print(f'Village Road in Dark : {vr_dark:.2f}')

```

```

# Create DataFrame for the grouped bar plot
comparison_df = pd.DataFrame({
    'Category': ['National Highway\nDaylight', 'National
Highway\nDark',
                'Village Road\nDaylight', 'Village
Road\nDark'],
    'Avg_Casualties': [nh_day, nh_dark, vr_day,
vr_dark],
    'Group': ['National Highway', 'National Highway',
'Village Road', 'Village Road']
})

```

Bar Plot - All four combinations in one graph

```

plt.figure(figsize=(12, 6))
sns.barplot(x='Category',
y='Avg_Casualties',
hue='Group',
data=comparison_df,

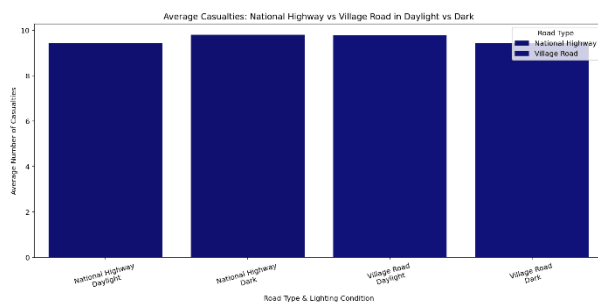
```

```
palette=['navy', 'darkblue', 'darkorange',
'chocolate'])
```

```
plt.title('Average Casualties: National Highway vs
Village Road in Daylight vs Dark')
plt.xlabel('Road Type & Lighting Condition')
plt.ylabel('Average Number of Casualties')
plt.legend(title='Road Type')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

Output:

```
National Highway in Daylight : 9.44
National Highway in Dark : 9.79
Village Road in Daylight : 9.78
Village Road in Dark : 9.44
```



F. DATA PREPARATION FOR MODELING

Before applying machine learning, numerical columns such as Driver Age and Speed Limit were normalized using MinMaxScaler. Accident location categories such as Straight Road, Curve, Intersection, and Bridge were encoded into numerical form. The dataset was then divided into training and testing sets using an 80:20 split ratio.

Code:

```
# Data Preparation for Modelling:
scaler = MinMaxScaler()
data[['Speed Limit (km/h)', 'Driver Age']] =
scaler.fit_transform(data[['Speed Limit (km/h)', 'Driver
Age']])
print(data[['Speed Limit (km/h)', 'Driver
Age']].describe())
```

Output:

	Speed Limit (km/h)	Driver Age
count	3000.000000	3000.000000
mean	0.499341	0.462315
std	0.297390	0.136413
min	0.000000	0.000000
25%	0.233333	0.373626
50%	0.500000	0.461538
75%	0.766667	0.560440
max	1.000000	1.000000

IV. Future Predictions – Machine Learning using CRISP-DM Process

Goal

The objective of this phase was to build a Linear Regression model for predicting the Number of Casualties using selected features.

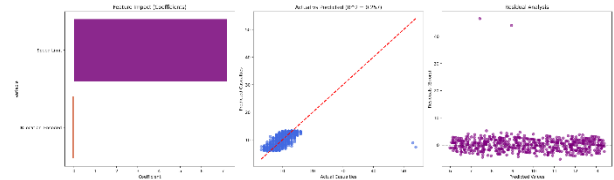
Code:

```
# (6.)Future Predictions: Machine Learning using the
CRISP-DM Process
# Focus: Supervised Learning (Linear Regression) and
CRISP-DM.
```

```
loc_map = {
    'Straight Road':1,
    'Curve':2,
    'Intersection':3,
    'Bridge':4
}
data['Location_Encoded'] = data['Accident Location
Details'].map(loc_map)
X = data[['Speed Limit (km/h)', 'Location_Encoded']]
Y = data['Number of Casualties']
X_train, X_test, Y_train, Y_test =
train_test_split(X, Y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, Y_train)

pred = model.predict(X_test)
mse = mean_squared_error(Y_test, pred)
r2 = r2_score(Y_test, pred)
```

```
print(f'Mean Squared Error: {mse:.4f}')
print(f'Model Accuracy (R-squared): {r2:.4f}')
print(f'Intercept: {model.intercept_:.4f}')
print(f'Location Coefficient: {model.coef_[1]:.4f}')
```



```
fig, axes = plt.subplots(1, 3, figsize=(20,6))
```

```
importance = pd.DataFrame({'Variable':['Speed
Limit','Location Encoded'],
                          'Coefficient': model.coef_})
```

```
sns.barplot(ax=axes[0], x='Coefficient', y='Variable',
data = importance,hue='Variable',
palette='plasma',legend=False)
axes[0].set_title('Feature Impact (Coefficients)')
```

```
# Actual vs Predicted
axes[1].scatter(Y_test, pred, alpha=0.6,
color='royalblue')
axes[1].plot([Y_test.min(), Y_test.max()],
[Y_test.min(), Y_test.max()],'r--',lw=2)
axes[1].set_title(f'Actual vs Predicted (R^2 = {r2:.3f})')
axes[1].set_xlabel('Actual Casualties')
axes[1].set_ylabel('Predicted Casualties')
```

```
residuals = Y_test - pred
axes[2].scatter(pred, residuals, alpha=0.6,
color='purple')
axes[2].axhline(y=0, color='black', linestyle='--')
axes[2].set_title('Residual Analysis')
axes[2].set_xlabel('Predicted Values')
axes[2].set_ylabel('Residuals (Errors)')
plt.tight_layout()
plt.show()
```

```
print(f'Speed Coefficient: {model.coef_[0]:.4f}')
```

Output:

```
Mean Squared Error: 10.7141
Model Accuracy (R-squared): 0.2572
Intercept: 6.2583
Location Coefficient: -0.0690
Speed Coefficient: 7.2034
```

ACKNOWLEDGMENT

The author expresses sincere gratitude to the faculty of the Department of Computer Science and Engineering at Lovely Professional University for their guidance and support throughout the completion of this project. The author would also like to thank the Government of India for providing access to the air quality dataset through [Kaggle](#), which served as the foundation for this analysis. Additionally, appreciation is extended to the Python open-source community for developing and maintaining libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn, which were extensively used in this project..

REFERENCES

- [1] Wes McKinney, *Python for Data Analysis*, O'Reilly Media.
- [2] Scikit-learn Official Documentation.
- [3] Pandas Official Documentation.
- [4] Matplotlib Official Documentation.
- [5] Seaborn Statistical Visualization Library.
- [6] SciPy Statistical Functions Documentation. *Sci. Eng.*, 2007.